CS-107 Lecture 12
Introduction to Low Level Languages

Troy Astarte

Megan Venn-Wycherley

Recap: the stored-program machine

- **One memory, both contents:** instructions and data are stored together, so a machine is programmed by loading a program rather than rewiring it.
- **Five parts:** input, output, memory, the arithmetic and logic unit, and the control unit.
- **The CPU** is the control unit and the ALU together, with registers holding the values in use.
- **Three buses connect them:** address, data and control.

Recap: the fetch-decode-execute cycle

- **Every instruction takes three steps:** the control unit fetches it from memory, decodes it, then executes it, and repeats.
- **Registers do the carrying:** the program counter holds the address of the next instruction, the address and data registers move it, and the instruction register holds it while it is decoded.
- **Execution uses the ALU,** with the result left in the accumulator or written back to memory.
- **Worked example:** LOAD 200, ADD 201, STORE 202, HALT — four instructions, eight passes through the cycle.

Recap: the von Neumann bottleneck

- **The bottleneck:** instructions and data cross the same bus, so the processor spends cycles waiting for memory.
- **Cache** holds recently used values close to the CPU; registers hold the ones in use.
- **The Harvard architecture** separates instruction memory from data memory, giving each its own bus.
- **Pipelining and multiple cores** overlap instructions instead of widening the bus.

Recap: memory and storage

- **Primary memory:** RAM is volatile and holds whatever is running; ROM is fixed and holds start-up code. Only primary memory is reachable directly by the processor.
- **The hierarchy:** access time rises and cost per byte falls as you move outwards from registers to cache, RAM, solid state and magnetic disk.
- **Magnetic disks** are addressed by track, sector and cylinder, and read by a head on a moving arm, so access time depends on seek and rotation.
- **Solid state drives** have no moving parts, so access time does not depend on where the data sits.

Recap: processing, and what comes next

- **Word size** is the number of bits the processor handles in one operation, which sets how much work a single instruction does.
- **Parallelism at four levels:** bit, instruction, data and task. Cores give task-level parallelism; a GPU gives data-level parallelism.
- **A specification sheet lists these properties:** cores, clock speed, cache, memory capacity and storage type.
- **Next: the instructions themselves.** Low-level languages, machine code and assembly, are what the control unit actually fetches and decodes.

Goals for Today

- What machine language is, and why a CPU only runs binary
- The Pep/9 virtual computer: its registers, memory and instruction set
- Instruction format: the instruction specifier and the operand specifier
- Addressing modes: immediate and direct
- Reading and tracing Pep/9 instructions, including input and output
- Two worked programs: printing "Hello World" and adding 5 + 4

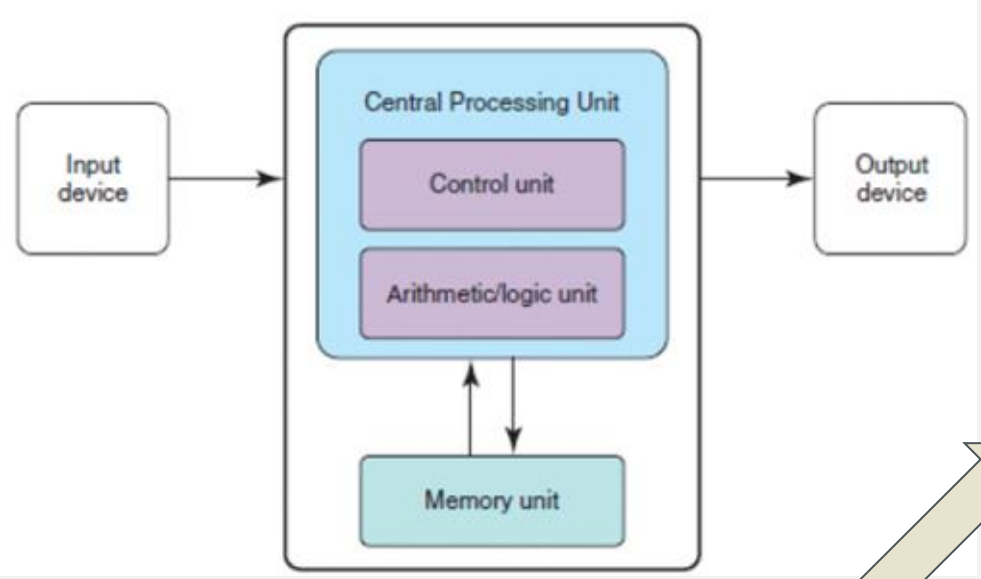
A Brief Recap

A brief recap

Computer:

- A programmable electronic device that can store, retrieve, and process data
- **Data** and **instructions** to manipulate the data are logically the same and can be stored in the same place. (Stored-Program concept)

A brief recap



Address	Contents
00000000	11100011
00000001	10101001
:	:
.	.
11111100	00000000
11111101	11111111
11111110	10101010
11111111	00110011

Fetch

Decode

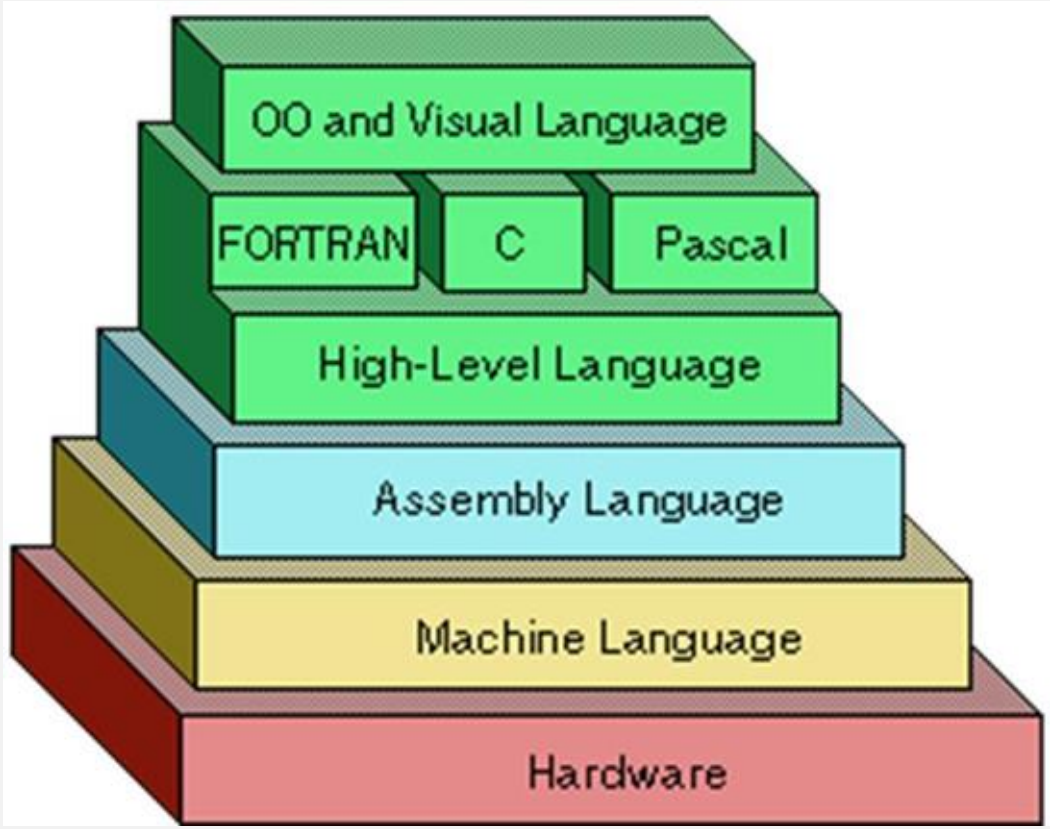
Execute

A brief recap

- **Abstraction**

The concept of hiding away the complicated details to allow us to focus on the key details we are interested in.

This also applies in programming languages, with lower-level and higher-level languages.

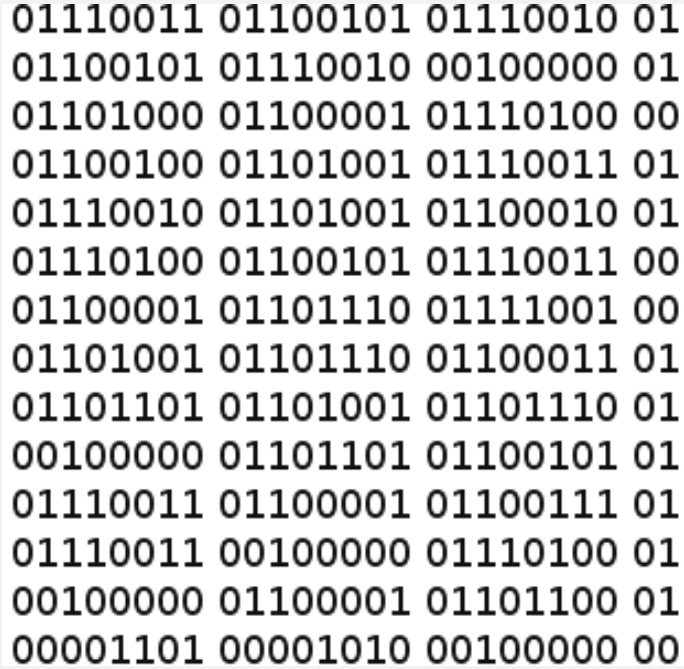


Machine Language

- **Machine language**

A language made up of binary coded instructions built into the hardware of a particular computer and used directly by the computer.

- Why would anyone choose to use machine language?
- (Hint: they had no choice. Why?)



Example machine code
(Disclaimer: I have no idea what this does)

Machine Language

- **Characteristics** of machine language:
  - Every processor type can have its own specific set of machine instructions.
  - The digital logic of the CPU recognizes the binary representations of the instructions
  - Each machine-language instruction does only one (typically) very low-level task

Pep/9

Pep/9 Virtual Computer

- **Virtual computer**

A hypothetical machine designed to contain the important features of a real computer that we want to illustrate

- **Pep/9** http://computersystemsbook.com/5th-edition/pep9/

A virtual computer designed by Stanley Warford that has 40 machine-language instructions.

The architecture uses a 16-bit processor.

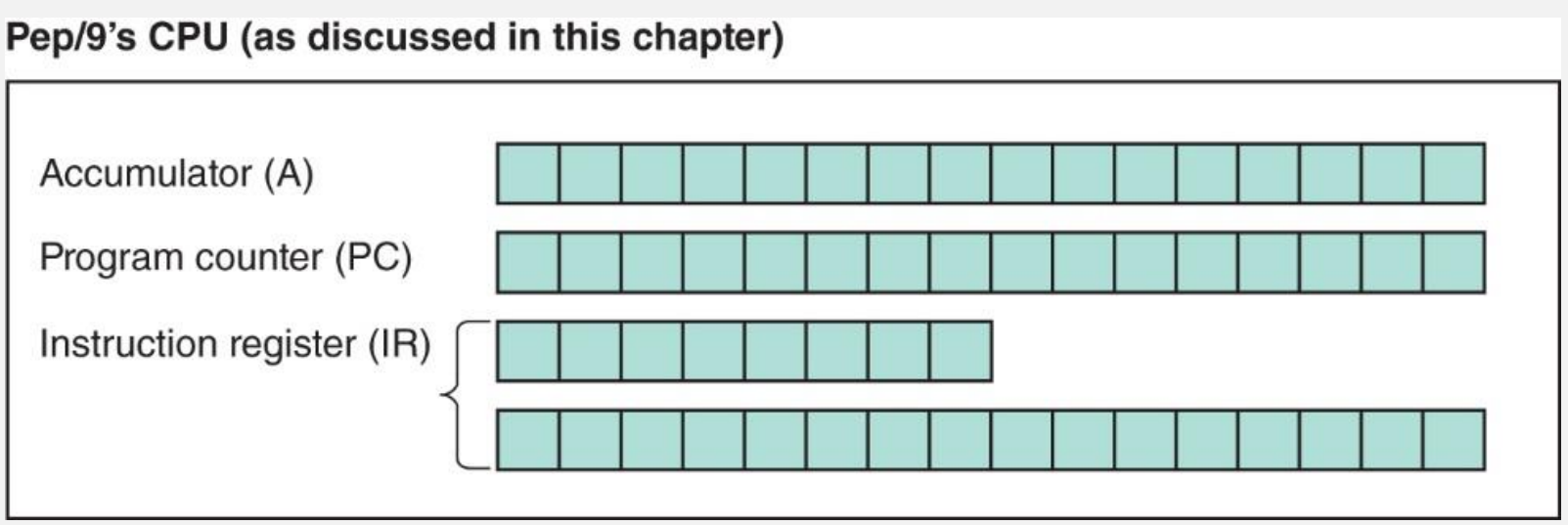
Pep/9 Features

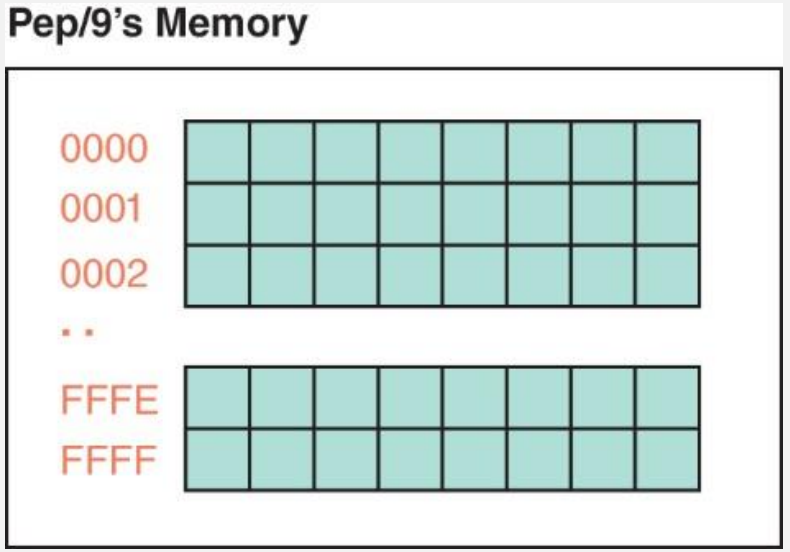
Pep/9 Registers/Status Bits:

- **Program counter** ("PC") (containing address of the next instruction to be executed)
- **Instruction register** ("IR") (contains a copy of the instruction being executed)
- **Accumulator** ("A") (used to hold data and results of operations)

- The main memory unit is made up of 64KB of storage, with single-byte locations.

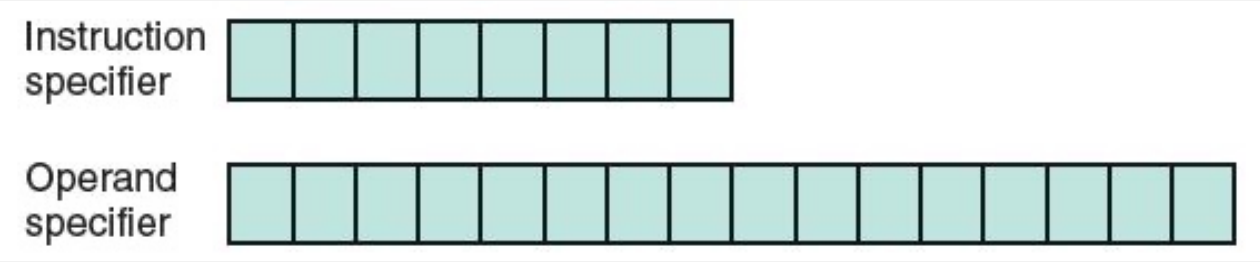
Pep/9 Architecture



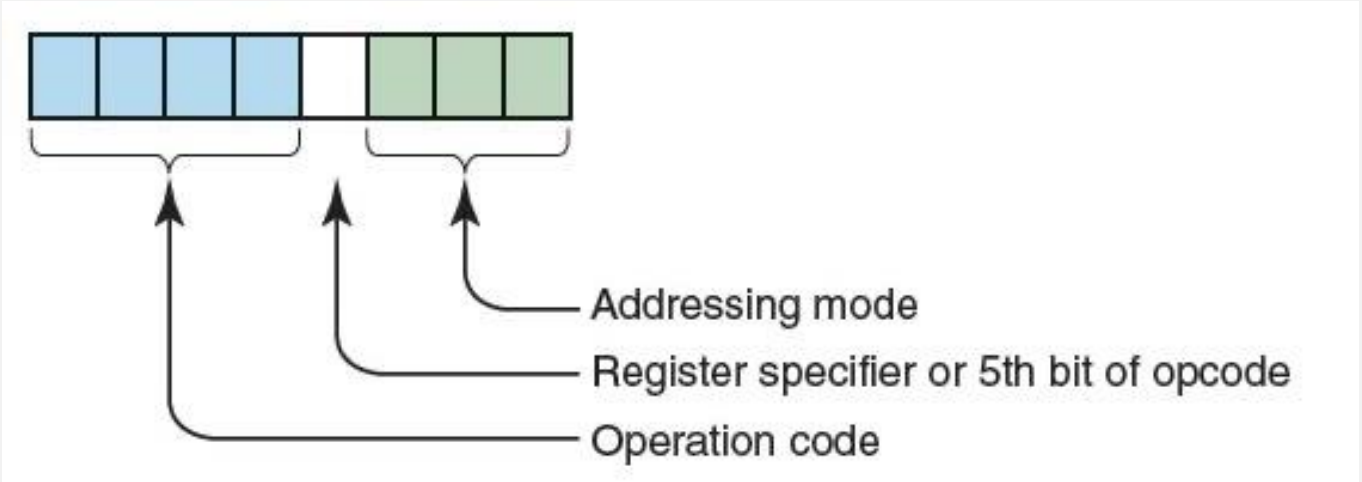


Instruction Format

- Each instruction is (optionally) made up of two parts:



- The Instruction specifier is also divided as follows:



Instruction Format

- **Operation code**

Specifies which instruction is to be carried out

- **Register specifier**

Specifies which register is to be used (for our purposes it always specifies the accumulator or 0)

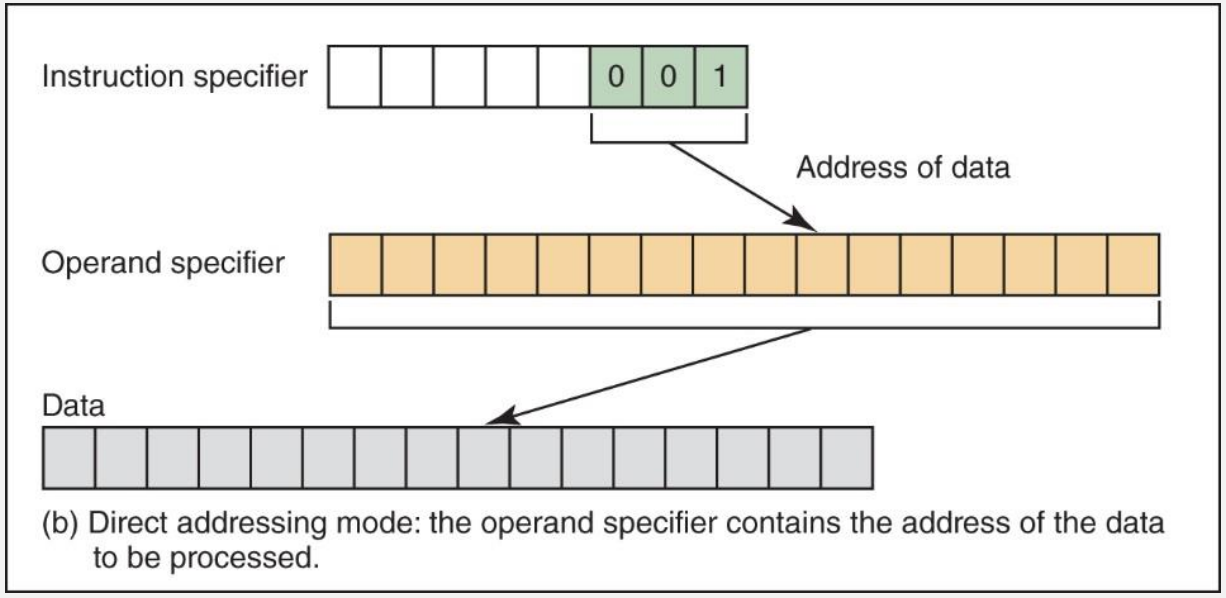
- **Addressing-mode specifier**

Says how to interpret the operand part of the instruction

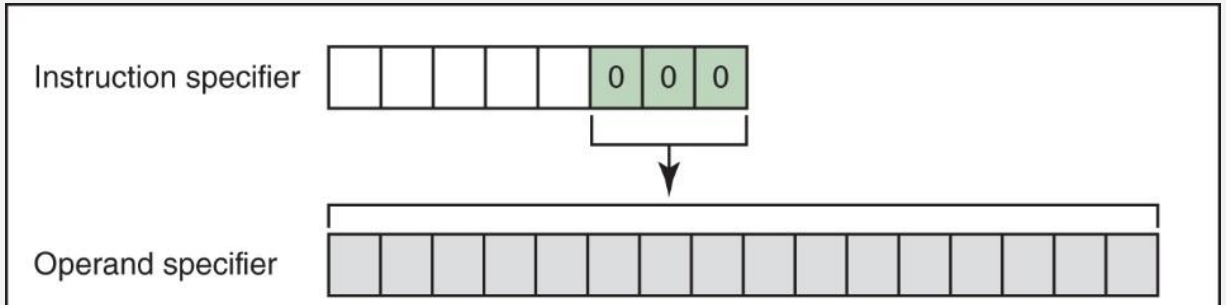
Instruction Format

- **Addressing-mode**

Indicates whether value is stored in operand specifier or memory

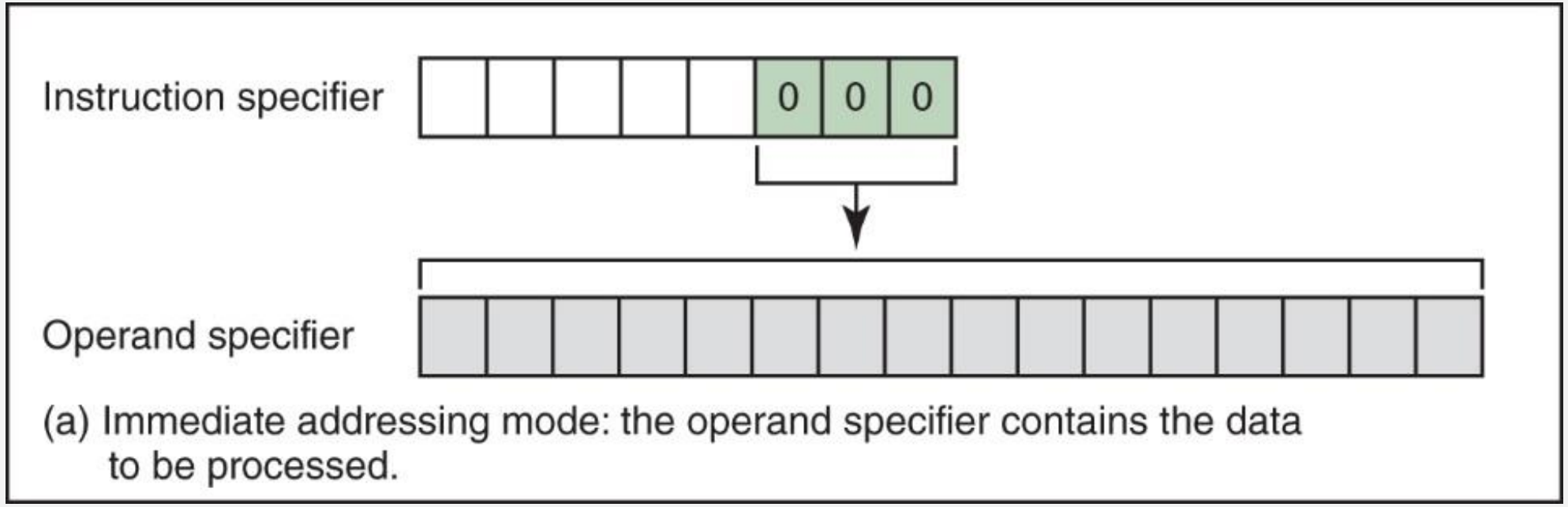


(b) Direct addressing mode: the operand specifier contains the address of the data to be processed.



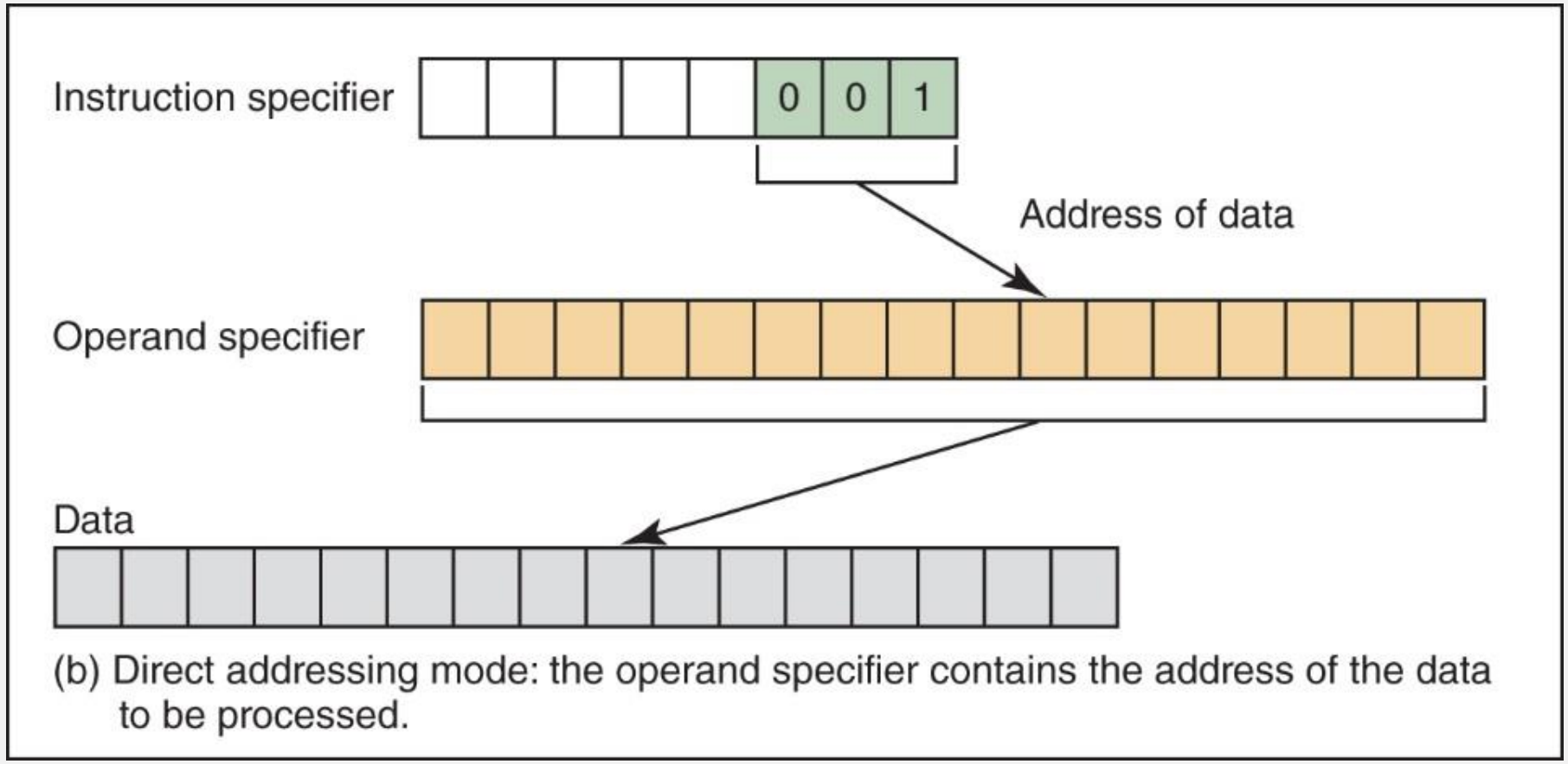
(a) Immediate addressing mode: the operand specifier contains the data to be processed.

Addressing modes: Immediate



(a) Immediate addressing mode: the operand specifier contains the data to be processed.

Addressing modes: Direct



(b) Direct addressing mode: the operand specifier contains the address of the data to be processed.

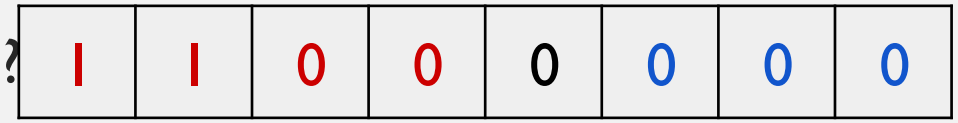
Sample Pep/9 Instructions

Some sample Pep/9 Instructions

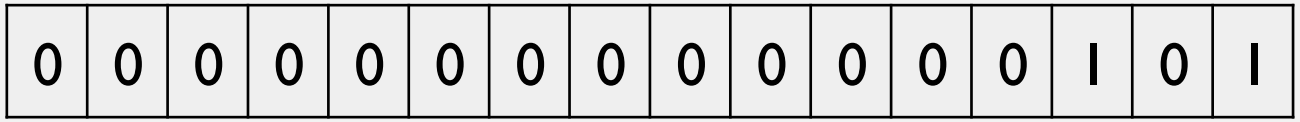
Opcode	Meaning of Instruction
0000	Stop execution
1100	Load word into the A register
1101	Load byte into the A register
1110	Store word from the A register
1111	Store byte from the A register
0110	Add the operand to the A register
0111	Subtract the operand from the A register

Some sample Pep/9 Instructions

What does the following do?

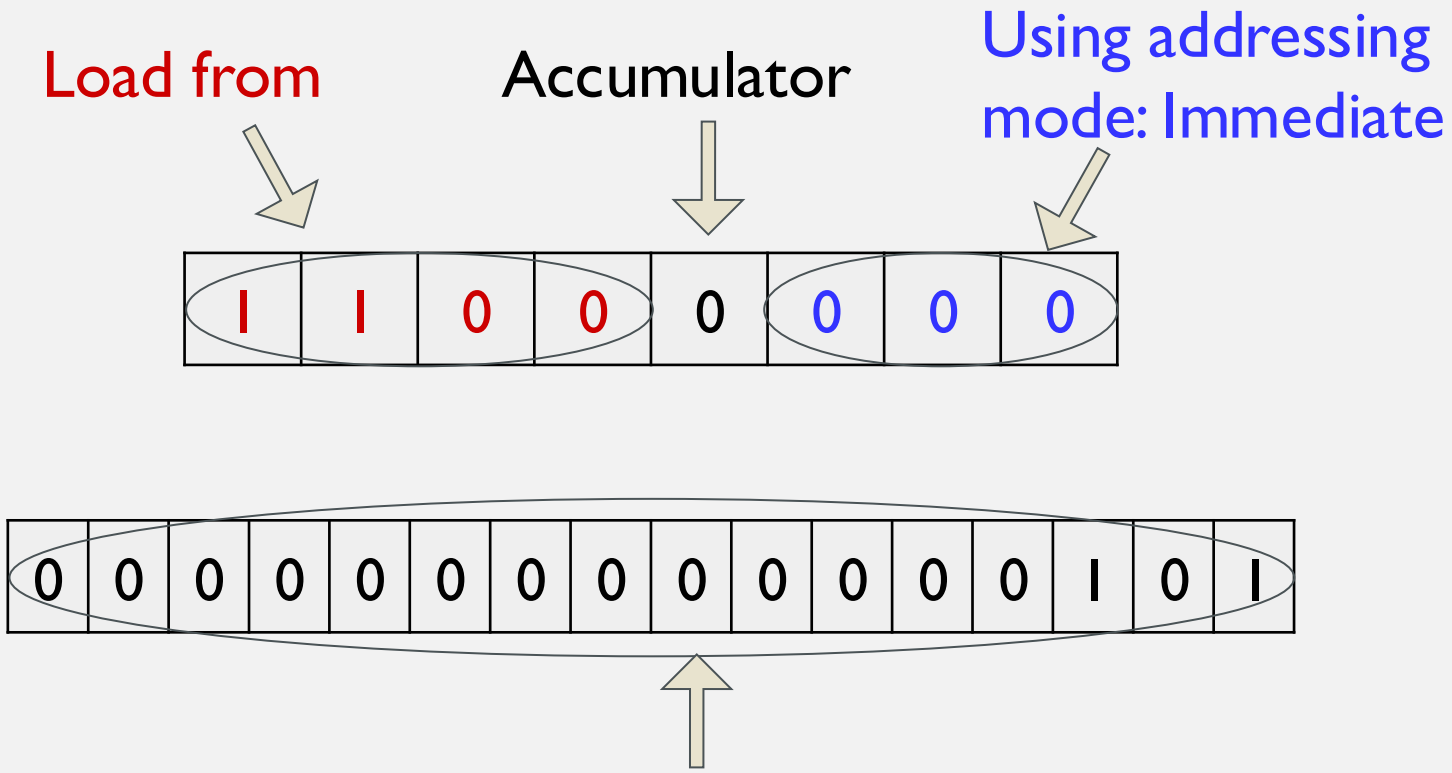


- Instruction Specifier:
- Operand Specifier:



- This loads the binary number 101 (i.e. 5) into the accumulator.

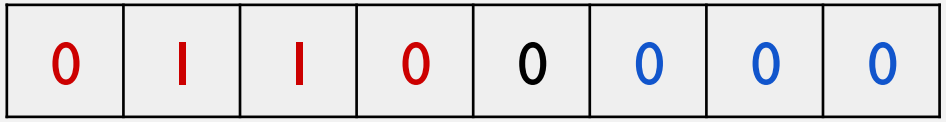
Some sample Pep/9 Instructions



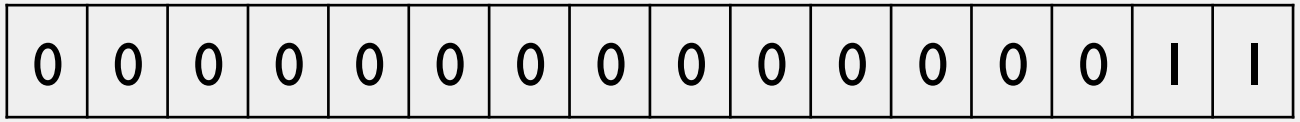
Because the addressing mode is immediate, this is the binary value loaded into the accumulator, i.e. decimal 5

Some sample Pep/9 Instructions

What does the following do?

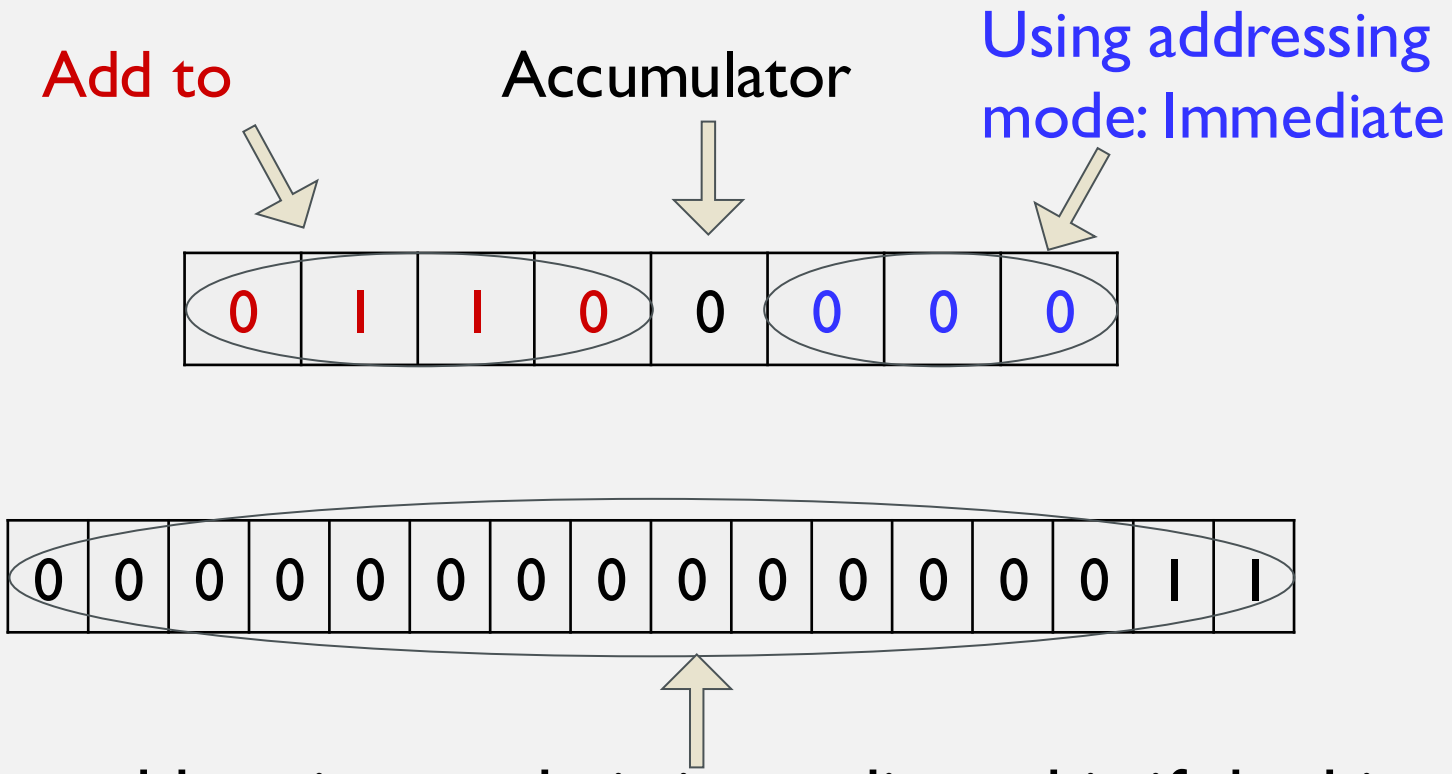


- Instruction Specifier:
- Operand Specifier:



- This adds the binary number 11 (i.e. 3) from the value currently in the accumulator.

Some sample Pep/9 Instructions



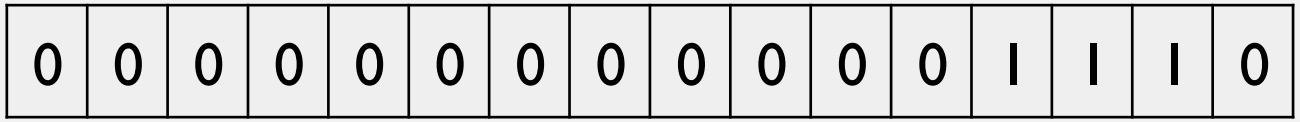
Because the addressing mode is immediate, this if the binary value added to the accumulator, i.e. decimal 3

Some sample Pep/9 Instructions

What does the following do?

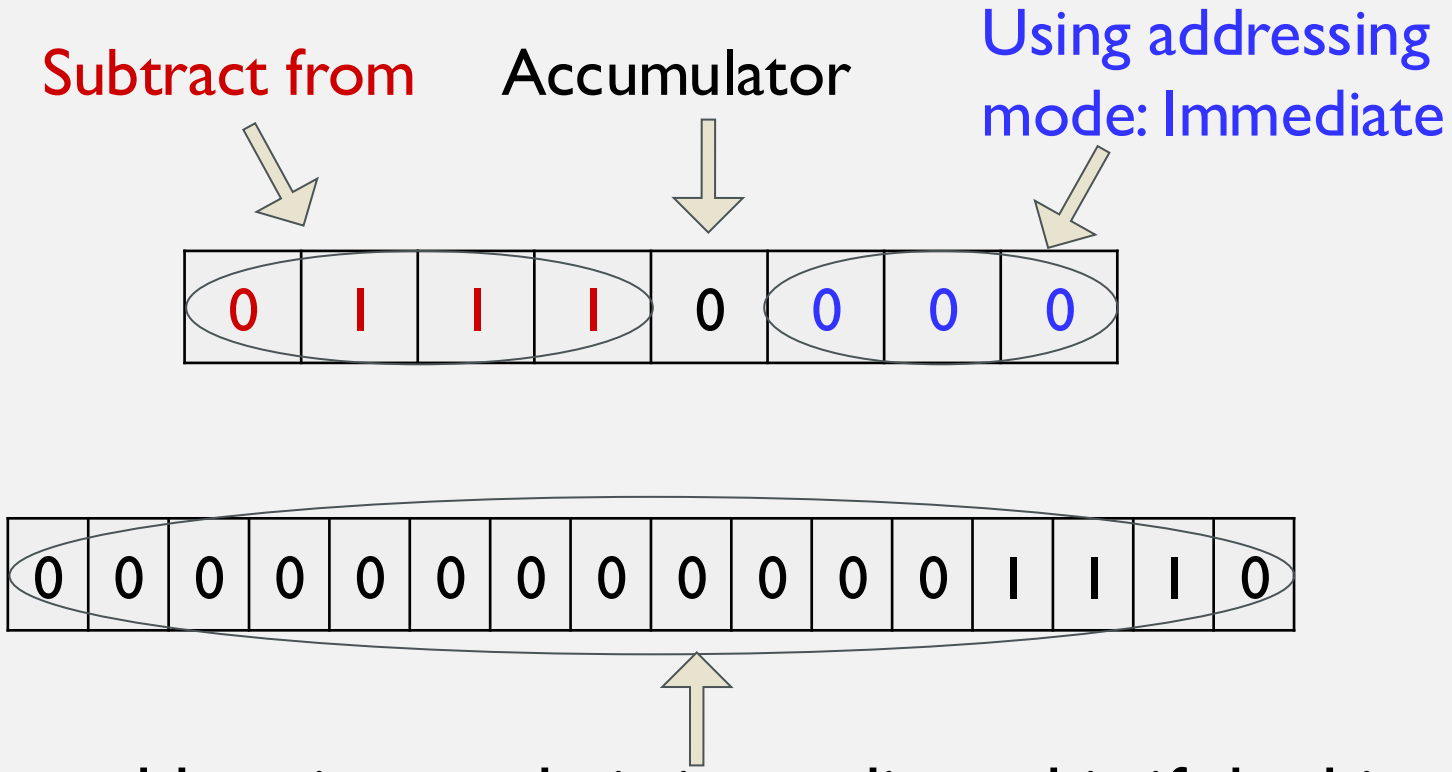


- Instruction Specifier:
- Operand Specifier:



- This subtracts the binary number 1110 (i.e. 14) from the value currently in the accumulator.

Some sample Pep/9 Instructions



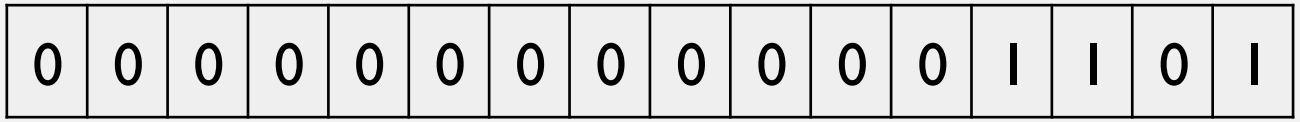
Because the addressing mode is immediate, this if the binary value subtracted from the accumulator, i.e. decimal 14

Some sample Pep/9 Instructions

What does the following do?

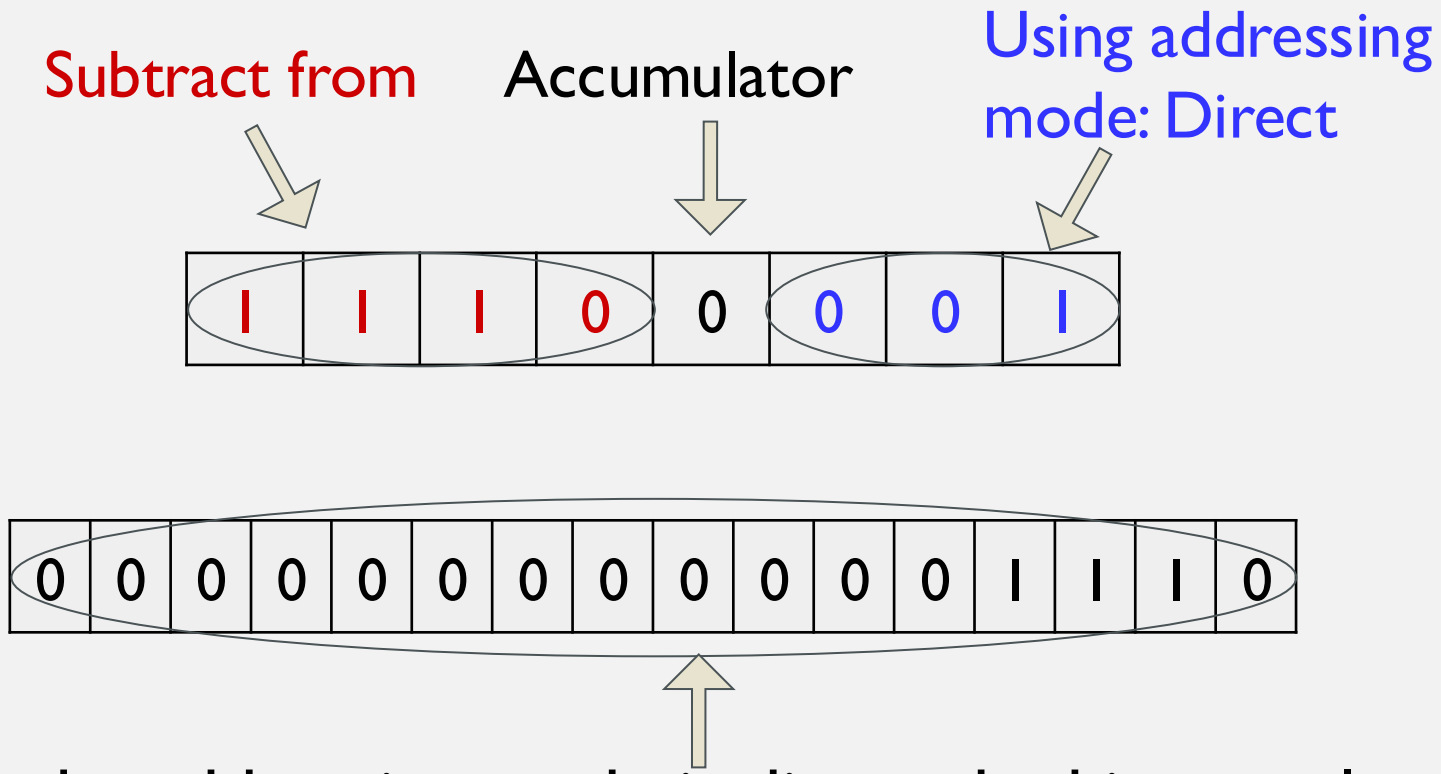


- Instruction Specifier:
- Operand Specifier:



- This stores the word from the accumulator into memory address 1101.

Some sample Pep/9 Instructions



Because the addressing mode is direct, the binary value in the operand specifier is used as a memory location.

Input and Output in Pep/9

- Need to get information in and out of our machine
- We can use **memory-mapped I/O**, where our input and output devices are connected to physical memory locations
- In Pep/9:
  - Input device is at location **FC15**
  - Output device is at location **FC16**

Input and Output in Pep/9

To get input in:
Load the value within FC15 to the accumulator

To get output out:
Load a value into the accumulator, and then store it to FC16

Note that both of these only accept a single byte of information!

Hello World in Pep/9

Example Program 1: Print "Hello World"

Let's print the classic "Hello World" string to the output device.

What would we need to do?

Steps required:

1. Load the letter H into the accumulator.
2. Store the value in the accumulator to the output device.
3. Load the letter e into the accumulator.
4. …

Example Program 1: Print “Hello World”

```
11000000000000000001001000
11110001111111000010110
11000000000000000001100101
11110001111111000010110
11000000000000000001101100
11110001111111000010110
11000000000000000001101100
11110001111111000010110
11000000000000000001101111
11110001111111000010110
11000000000000000100000
11110001111111000010110
11000000000000000001110111
11110001111111000010110
11000000000000000001101111
11110001111111000010110
11000000000000000001110010
11110001111111000010110
11000000000000000001101100
11110001111111000010110
11000000000000000001100100
11110001111111000010110
0000 0000
```

```
11000000000000000100100011110001111111000001011011000000000000
00011001011111000111111100000101101100000000000000011011001111
00011111110000010110110000000000000001101100111100011111110000
01011011000000000000001101111111100011111110000010110110000000
00000000010000011110001111111000001011011000000000000000011101
11111100011111110000010110110000000000000001101111111000111111
10000010110110000000000000011100101111000111111100000101101100
00000000000000011011001111000111111100000101101100000000000000
11001001111000111111100000101100000000
```

Example Program 1: Print "Hello World"

```
C0 00 48
F1 FC 16
C0 00 65
F1 FC 16
C0 00 6C
F1 FC 16
C0 00 6C
F1 FC 16
C0 00 6F
F1 FC 16
C0 00 20
F1 FC 16
C0 00 77
F1 FC 16
C0 00 6F
F1 FC 16
C0 00 72
F1 FC 16
C0 00 6C
F1 FC 16
C0 00 64
F1 FC 16
00
```

C0 00 48 F1 FC 16
C0 00 65 F1 FC 16
C0 00 6C F1 FC 16
C0 00 6C F1 FC 16
C0 00 6F F1 FC 16
C0 00 20 F1 FC 16
C0 00 77 F1 FC 16

Example Program 1: Print "Hello World"

H = entry 72, which is 48_{16}

e = entry 101, which is $6C_{16}$

…

Left Digit(s) \ Right Digit	0	1	2	3	4	5	6	7	8	9
	ASCII									
0	NUL	SOH	STX	ETX	EOT	ENQ	ACK	BEL	BS	HT
1	LF	VT	FF	CR	SO	SI	DLE	DC1	DC2	DC3
2	DC4	NAK	SYN	ETB	CAN	EM	SUB	ESC	FS	GS
3	RS	US	□	!	"	#	$	%	&	'
4	(	)	*	+	,	-	.	/	0	1
5	2	3	4	5	6	7	8	9	:	;
6	<	=	>	?	@	A	B	C	D	E
7	F	G	H	I	J	K	L	M	N	O
8	P	Q	R	S	T	U	V	W	X	Y
9	Z	[	\	]	^	_	`	a	b	c
10	d	e	f	g	h	i	j	k	l	m
11	n	o	p	q	r	s	t	u	v	w
12	x	y	z	{	|	}	~	DEL		

Example Program 2: 5 + 4

Let's try to add the numbers 4 and 5 together and store the result in memory address 00001100.

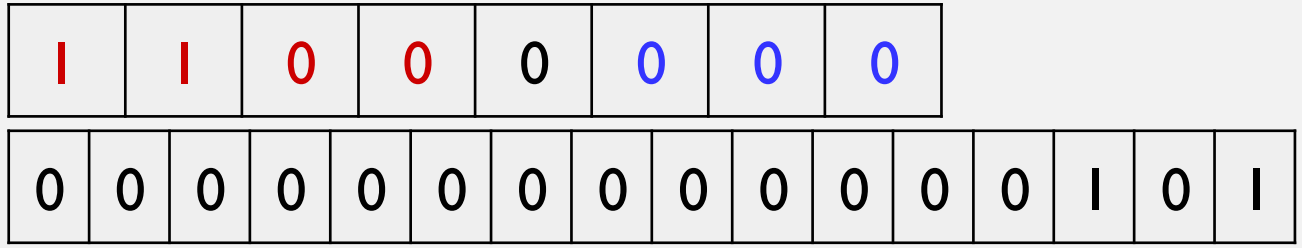
What would we need to do?

Steps required:

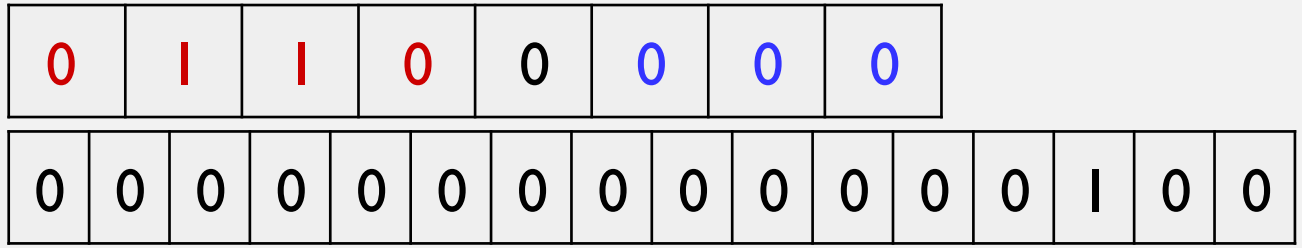
1. Load the number 5 into the accumulator.
2. Add the number 4 to the accumulator.
3. Store the answer (the value held in the accumulator into memory location 00001100).

Example Program 2: 5 + 4

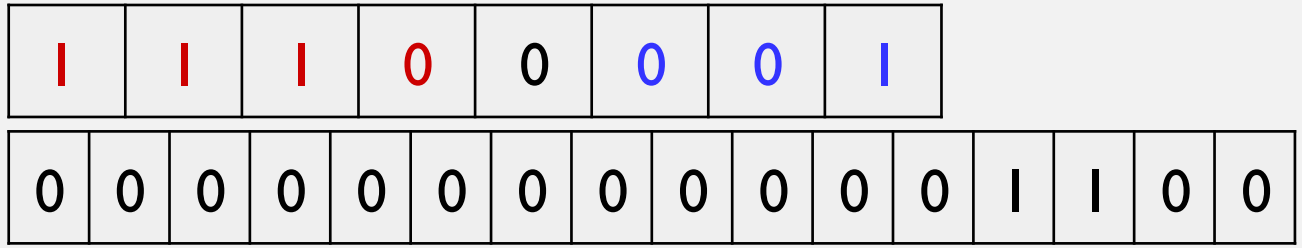
Load number 5 into accumulator:



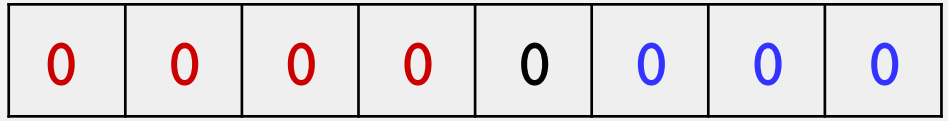
Add number 4 to the accumulator:



Store the result:



Stop the program:



Example Program 2: 5 + 4

- Full program:

```
11000000 0000000000000101
01100000 0000000000000100
11100001 0000000000001100
00000000 0000000000000000
```

Or in Hexadecimal:

C00005 600004 E1000C 000000

Goals for Today

- What machine language is, and why a CPU only runs binary
- The Pep/9 virtual computer: its registers, memory and instruction set
- Instruction format: the instruction specifier and the operand specifier
- Addressing modes: immediate and direct
- Reading and tracing Pep/9 instructions, including input and output
- Two worked programs: printing "Hello World" and adding 5 + 4

CS-107 Lecture 12
Introduction to Low Level Languages

Troy Astarte

Megan Venn-Wycherley